

Universidad de León

Ingeniería Informática

Sistemas de información y Business Intelligence

TVSHOW RECOMMENDER

José Benito Edú NGOMO MEDJA

Índice

Introducción.....	4
Contexto y planteamiento del problema.....	4
Motivación y objetivos del proyecto	5
Alcance y limitaciones iniciales.....	5
Fundamentación y requisitos	6
Necesidades detectadas	6
Enfoque general de la solución.....	7
Requisitos funcionales y no funcionales.....	7
Recursos y fuentes de información utilizadas	9
Tecnología y arquitectura	10
Tecnologías, herramientas y entornos de desarrollo	10
Justificación de las herramientas seleccionadas	11
Arquitectura general del sistema.....	12
Componentes principales y flujo interno de la aplicación	12
Desarrollo de la solución	14
Diseño e implementación	14
Explicación detallada del funcionamiento del sistema	24
Interfaz de acceso y elementos clave para la demostración	26
Consideraciones de ciberseguridad.....	30
Nivel de seguridad alcanzado	30
Medidas recomendadas para despliegue en servidor	31
Manual de usuario.....	32
Instalación y configuración	32
Inicio y navegación por la aplicación	34
Funcionalidades principales	34
Validación.....	35
Casos de uso	36
Escenario favorable.....	36

Escenario desfavorable	36
Escenario realista	37
Resultados obtenidos	37
Qué permite hacer la aplicación y qué no	38
Evaluación crítica	39
Análisis DAFO	39
Limitaciones técnicas detectadas	40
Posibles mejoras	41
Lecciones aprendidas	41
Aplicabilidad práctica futura	41
Conclusiones	42
Bibliografía	43

Introducción

La industria del entretenimiento digital ha evolucionado desde un modelo de escasez (televisión lineal), a uno de sobreabundancia (streaming bajo demanda). Este cambio ha traído consigo, volúmenes masivos de datos y contenidos dispersos. El presente nace como respuesta tecnológica a esta nueva realidad. A continuación se detalla la problemática que justifica su desarrollo, los objetivos marcados y el alcance de la solución propuesta.

Contexto y planteamiento del problema

Actualmente el ecosistema del *streaming* sufre una crisis de fragmentación sin precedentes. Lo que comenzó con Netflix como actor dominante se ha transformado en una "Guerra del Streaming" donde cada productora (Disney, Amazon, Hulu, HBO, etc.) ha creado su propio recinto amurallado. Esta dispersión del contenido ha generado una experiencia de usuario rota y frustrante caracterizada por tres problemas críticos que hacen necesaria una herramienta unificadora:

1. **Ineficiencia y "Fatiga de Aplicaciones":** El usuario promedio debe saltar constantemente entre 3 o 4 aplicaciones diferentes solo para responder a la pregunta "¿qué veo hoy?". No existe un punto centralizado de verdad. Buscar una película implica abrir una app, buscar, cerrar, abrir la siguiente y repetir el proceso. Esta fricción tecnológica consume tiempo valioso y a menudo lleva al usuario a desistir o volver a ver contenido repetido por pura comodidad.
2. **La Paradoja de la Elección:** Psicológicamente, el exceso de opciones mal organizadas paraliza la toma de decisiones. Los catálogos suman decenas de miles de títulos, pero las herramientas de navegación de las plataformas son primitivas. Se limitan a mostrar "lo más popular" o "novedades", enterrando joyas ocultas que podrían ser perfectas para el usuario pero que quedan invisibles bajo algoritmos comerciales.
3. **La Limitación de los Buscadores Actuales:** Si un usuario busca *"una serie policíaca oscura como True Detective pero con toques de ciencia ficción"*, los buscadores tradicionales de las plataformas fallan estrepitosamente. Funcionan por palabras clave literales (título, actor), no por **conceptos ni contexto semántico**. Las plataformas actuales no "entienden" el contenido, solo lo indexan.

Por tanto, existe una necesidad técnica no resuelta. Hace falta una capa de abstracción superior. Una herramienta que se sitúe *por encima* de las plataformas, que rompa los silos de datos, y permita al usuario dialogar con el catálogo global como si le

preguntara a un experto cinéfilo, en lugar de teclear palabras clave en una barra de búsqueda inerte.

Motivación y objetivos del proyecto

La motivación principal es demostrar que la tecnología actual (específicamente los Grafos de Conocimiento y los LLMs) ya está madura para solucionar este problema de fragmentación. No se trata solo de crear un "agregador", sino de dotarlo de inteligencia. La motivación es pasar de un modelo de **búsqueda pasiva** en el usuario busca, a uno de **descubrimiento activo y conversacional**, en el que el sistema entiende y propone.

Para cubrir esta necesidad, se plantean los siguientes objetivos:

1. **Unificar la Oferta:** Consolidar en una única base de datos los catálogos dispares de Netflix, Amazon Prime, Hulu y Disney+.
2. **Habilitar la Búsqueda Semántica:** Implementar vectores de incrustación (*embeddings*) sobre las sinopsis permitiendo encontrar similitudes temáticas profundas (el "tono" o la "atmósfera" de la serie) que escapen a los filtros tradicionales.
3. **Interfaz Conversacional (RAG):** Desarrollar un sistema donde el usuario utilice lenguaje natural para pedir recomendaciones y recibir explicaciones lógicas generadas por IA sobre por qué se le sugiere ese título.

Alcance y limitaciones iniciales

El proyecto abarca el desarrollo "*End-to-End*" de un prototipo funcional. Esto incluye la ingeniería de datos (limpieza y fusión de CSVs), la arquitectura del backend (API con FastAPI y base de datos en grafo), y la integración de modelos de IA (Llama 3 vía Groq y LlamaIndex). El sistema final permitirá al usuario realizar consultas complejas sobre series de TV de las cuatro grandes plataformas mencionadas y recibir respuestas contextualizadas.

Limitaciones Iniciales: Es importante acotar que en esta primera fase:

- **Volatilidad de los Datos:** El sistema trabaja con *datasets* estáticos importados, y no con conexiones en tiempo real a las plataformas. Por lo tanto, no reflejará cambios de catálogo diarios como bajas o estrenos de última hora.

- **Sin Historial de Usuario:** Al no tener usuarios reales previos, el sistema no utiliza filtrado colaborativo (no sabe "*qué ven otros usuarios parecidos a ti*"), sino que se basa puramente en el contenido y la semántica de las series.
- **Restricción de Formato:** El alcance se limita a Series de TV (*TV Shows*) excluyendo de momento películas y documentales para asegurar la calidad y consistencia de los datos en el grafo durante el desarrollo del prototipo.

Fundamentación y requisitos

El desarrollo de cualquier sistema de software eficaz requiere establecer un puente sólido entre la problemática abstracta y la implementación técnica. En este capítulo se definen las necesidades concretas que el sistema debe satisfacer y se traducen en requisitos formales que guiarán el desarrollo de la aplicación.

Necesidades detectadas

Tras el análisis del contexto actual del mercado de *streaming*, se han identificado tres necesidades críticas que el sistema debe resolver para aportar valor al usuario final:

1. **Necesidad de Centralización:** El usuario carece de una herramienta unificada. Se detecta la necesidad de agregar metadatos de múltiples fuentes heterogéneas (Netflix, Amazon, Hulu, Disney+) en un repositorio común eliminando la necesidad de consultar múltiples aplicaciones.
2. **Necesidad de Búsqueda Semántica:** Los motores de búsqueda actuales basados en palabras clave son insuficientes para consultas abstractas. El usuario necesita poder expresar intenciones complejas (e.g., "*algo con la atmósfera de los años 80 pero de terror*") y que el sistema interprete el significado latente a parte del literalidad.
3. **Necesidad de Interacción Natural:** Existe una barrera en las interfaces de filtrado tradicionales (checkboxes, desplegables). Se requiere una interfaz conversacional (chatbot) que emule una recomendación humana capaz de gestionar tiempos de espera y ofrecer una experiencia de usuario fluida mientras se procesan consultas complejas en el servidor.

Enfoque general de la solución

Para cubrir estas necesidades se propone una arquitectura web moderna basada en el paradigma **cliente-servidor** apoyada por técnicas de Inteligencia Artificial Generativa.

- **Backend (Servidor):** Se plantea un núcleo desarrollado en **Python** que orquesta la lógica de negocio. Este núcleo interactúa con una base de datos orientada a grafos (**Neo4j**) para las relaciones entre contenidos y utiliza modelos de lenguaje (**LLMs** vía Groq y Ollama) para la comprensión semántica. Se expone mediante una API RESTful.
- **Frontend (Cliente):** Se opta por una interfaz web ligera y desacoplada construida con estándares web nativos (HTML5, CSS3, Vanilla JS). El diseño prioriza la simplicidad visual y la retroalimentación inmediata al usuario mediante animaciones, gestionando la asincronía de las respuestas de la IA.

Requisitos funcionales y no funcionales

A continuación, se detallan los requisitos del sistema separando las responsabilidades entre el cliente (navegador) y el servidor (API).

Requisitos Funcionales

Del lado del Cliente (Frontend):

- **RF-C-01 (Envío de Consultas):** La interfaz debe permitir al usuario introducir texto natural y enviarlo al servidor mediante una petición HTTP POST segura.
- **RF-C-02 (Gestión de Estado de Espera):** Dado que la inferencia de IA puede tardar varios segundos, el cliente debe mostrar una animación de "*escribiendo*" (typing dots) inmediatamente después de enviar el mensaje para informar al usuario de que el sistema está trabajando.
- **RF-C-03 (Mecanismo de Polling):** El cliente debe implementar un sistema de sondeo (polling). Tras enviar el mensaje, deberá consultar periódicamente (cada 2 segundos aproximadamente) el endpoint de resultados proporcionado por el servidor hasta recibir el estado ready.
- **RF-C-04 (Renderizado de Respuestas):** El sistema debe mostrar el texto recibido con un efecto de "mecanografiado" (aparición palabra por palabra) para mejorar la lectura y naturalidad de la conversación.

- **RF-C-05 (Control de Interfaz):** Se debe deshabilitar el botón de envío mientras se espera una respuesta para evitar peticiones duplicadas o condiciones de carrera.
- **RF-C-06 (Auto-scroll Inteligente):** El chat debe desplazarse automáticamente hacia abajo cuando llegan nuevos mensajes, salvo que el usuario haya subido manualmente para leer el historial.

Del lado del Servidor (Backend):

- **RF-S-01 (Recepción Asíncrona):** El servidor debe exponer un endpoint (/send) que acepte la consulta, inicie el proceso de IA en segundo plano y devuelva inmediatamente un identificador de tarea (link_id) para evitar el *timeout* del navegador.
- **RF-S-02 (Orquestación RAG):** El sistema debe convertir la consulta del usuario en *embeddings* vectoriales y buscar nodos relevantes en la base de datos Neo4j basándose en similitud coseno.
- **RF-S-03 (Generación de Respuesta):** Utilizando la información recuperada del grafo, el servidor debe construir un *prompt* enriquecido y enviarlo al LLM (Groq) para generar la recomendación final.
- **RF-S-04 (Endpoint de Estado):** El servidor debe proveer un endpoint de consulta (/result/{link_id}) que devuelva el estado del proceso (not ready o ready) y el mensaje final una vez generado.

Requisitos No Funcionales

- **RNF-01 (Disponibilidad y Latencia):** El sistema debe ser capaz de gestionar la latencia inherente a los LLMs sin bloquear la interfaz del usuario.
- **RNF-02 (Usabilidad):** La interfaz de inicio debe presentar un diseño inmersivo (video de fondo con efectos de desenfoque) para atraer la atención del usuario antes de iniciar el chat.
- **RNF-03 (Compatibilidad):** El cliente web debe ser compatible con los principales navegadores modernos (Chrome, Firefox, Edge) y adaptable a dispositivos móviles (*Responsive Design*), gestionando eventos táctiles (touchstart).

Recursos y fuentes de información utilizadas

Para la materialización de este proyecto, ha sido necesario combinar fuentes de datos masivas con recursos de conocimiento técnico avanzado y servicios de Inteligencia Artificial. A continuación, se detallan los recursos nucleares que componen la base del sistema:

1. Fuentes de Datos (Datasets) El dataset utilizado proviene de la plataforma **Kaggle** desde la cual se han extraído conjuntos de datos en formato CSV, que contienen los catálogos históricos de las principales plataformas de streaming. Estos *datasets* incluyen metadatos cruciales como títulos, sinopsis, reparto, año de lanzamiento y clasificación de:

- Netflix
- Amazon Prime Video
- Hulu
- Disney+

2. Recursos de Conocimiento y Documentación Técnica Dado el carácter innovador de la arquitectura RAG (*Retrieval-Augmented Generation*), se ha recurrido a documentación especializada para implementar correctamente el flujo de información:

- **Documentación de Neo4j y Graph Data Science:** Consultada para el modelado eficiente de grafos y la redacción de consultas en lenguaje Cypher, permitiendo estructurar las relaciones entre "Series", "Géneros" y "Plataformas".
- **Tutoriales y 'Papers' sobre RAG:** Investigación sobre patrones de diseño para sistemas de recuperación aumentada fundamental para entender cómo conectar una base de datos vectorial con un modelo de lenguaje.
- **Documentación de LlamaIndex y FastAPI:** Guías oficiales utilizadas para la orquestación de la ingesta de datos y el desarrollo de la API asíncrona.

3. Modelos y Motores de Inteligencia Artificial (Core) El núcleo inteligente de la aplicación se sustenta en recursos de IA de última generación:

- **Modelos de Lenguaje (LLMs):** Se ha utilizado **Llama 3**, un modelo avanzado de código abierto capaz de interpretar instrucciones complejas y generar texto coherente en español.

- **Motor de Inferencia Groq:** Recurso crítico que proporciona la potencia de computación (LPUs) necesaria para ejecutar el LLM a velocidades ultra-rápidas, permitiendo una experiencia de chat casi instantánea.
- **Ollama y Embeddings:** Herramienta utilizada para la gestión local de modelos y la generación de *embeddings* (representaciones vectoriales del texto), esenciales para que el sistema pueda realizar búsquedas por similitud semántica en las descripciones de las series.

Tecnología y arquitectura

Este capítulo describe el ecosistema tecnológico sobre el que se sustenta el sistema de recomendación detallando las herramientas de software seleccionadas, y la arquitectura lógica diseñada para integrar bases de datos en grafo con inteligencia artificial generativa.

Tecnologías, herramientas y entornos de desarrollo

Para la construcción del sistema se ha optado por un *stack* tecnológico moderno, priorizando lenguajes y librerías con fuerte soporte para Ciencia de Datos e Inteligencia Artificial.

Lenguajes de Programación:

- **Python (v3.x):** Lenguaje principal del *backend*. Seleccionado por su dominio absoluto en el ecosistema de IA y su facilidad para manipular datos.
- **JavaScript (ES6+):** Utilizado en el cliente (*frontend*) para la gestión asíncrona de peticiones y manipulación del DOM.
- **HTML5 / CSS3:** Para la estructuración y estilizado de la interfaz web.
- **Cypher:** Lenguaje de consulta declarativo propio de Neo4j utilizado para interactuar con la base de datos de grafos.

Frameworks y Librerías Principales:

- **FastAPI:** Framework web moderno y de alto rendimiento para construir APIs con Python.
- **LlamaIndex:** Framework de orquestación de datos para conectar fuentes de datos personalizadas (el grafo) con grandes modelos de lenguaje (LLMs).
- **Pandas:** Utilizado en la fase de ETL (Extract, Transform, Load) para la limpieza, fusión y manipulación de los archivos CSV de los catálogos.

- **Uvicorn:** Servidor ASGI para ejecutar la aplicación web de forma asíncrona.

Entornos de Desarrollo (IDEs):

- **PyCharm:** Utilizado para el desarrollo del backend debido a sus potentes herramientas de depuración y gestión de entornos virtuales Python.
- **Visual Studio Code (VSCode):** Empleado para el desarrollo del frontend por su ligereza y extensiones para desarrollo web.

Gestión de Datos y Versiones:

- **Neo4j (Desktop & Community Server):** Sistema gestor de base de datos orientado a grafos.
- **Git & GitHub:** Para el control de versiones y el repositorio remoto del código fuente.

Justificación de las herramientas seleccionadas

La elección de cada componente responde a criterios de eficiencia, escalabilidad y adecuación al problema:

1. **¿Por qué Neo4j en lugar de SQL?** Los sistemas de recomendación se basan intrínsecamente en **relaciones** (Usuario -> GUSTA -> Género -> CONTIENE -> Película). Las bases de datos relacionales (SQL) requieren *joins* costosos para navegar estas conexiones. Neo4j permite modelar estas relaciones como ciudadanos de primera clase, y, crucialmente, ofrece soporte nativo para **índices vectoriales**, lo que permite combinar búsqueda por grafos y búsqueda por similitud semántica en la misma consulta.
2. **¿Por qué FastAPI?** A diferencia de frameworks más pesados como Django o más antiguos como Flask, FastAPI es asíncrono por defecto (*async/await*). Esto es vital para nuestra arquitectura, ya que el servidor debe gestionar múltiples peticiones de chat que esperan respuestas de la IA (operaciones I/O bound) sin bloquear el hilo principal de ejecución.
3. **¿Por qué Groq y Llama 3?** El principal cuello de botella en las aplicaciones de Chatbot con LLMs es la latencia. La API de Groq, mediante el uso de LPUs (Language Processing Units), ofrece una velocidad de inferencia drásticamente superior a la media del mercado. Llama 3 ha sido seleccionado por ser uno de los modelos *open-source* más potentes en la actualidad, con una excelente capacidad de razonamiento en español.
4. **¿Por qué Vanilla JS (sin React/Vue)?** Dado que el requisito del frontend es una interfaz de chat única y minimalista, el uso de frameworks SPA (Single Page

Application) complejos añadiría una sobrecarga innecesaria. El uso de JavaScript nativo garantiza un código ligero, una carga rápida y un control total sobre el ciclo de vida de la animación de escritura y el *polling*.

Arquitectura general del sistema

El sistema sigue una arquitectura **Cliente-Servidor** desacoplada, implementando el patrón de diseño **RAG (Retrieval-Augmented Generation)**.

El diseño se divide en tres capas lógicas:

1. **Capa de Presentación (Frontend):** Responsable de la interacción con el usuario. No contiene lógica de negocio, simplemente captura la entrada (prompt) y visualiza la respuesta. Se comunica con el backend mediante una API REST.
2. **Capa de Aplicación y Lógica (Backend):** El núcleo del sistema. Aquí reside el servidor FastAPI que actúa como orquestador. Recibe la consulta, gestiona la sesión y coordina la comunicación entre la base de datos y el modelo de lenguaje.
3. **Capa de Datos y Conocimiento (Persistence & Intelligence):**
 - **Almacenamiento:** Neo4j guarda tanto la información estructurada (títulos, géneros) como los vectores (embeddings).
 - **Inferencia:** Groq/Llama 3 procesa el contexto recuperado y genera la respuesta final.

Componentes principales y flujo interno de la aplicación

Para comprender el funcionamiento interno, se detalla el flujo de datos desde que el usuario envía un mensaje, hasta que recibe respuesta. Este flujo es asíncrono para mejorar la experiencia de usuario:

Componentes Clave:

- **ETL Processor:** (Script script.ipynb) Módulo offline encargado de ingerir los CSVs, limpiar datos, generar embeddings con Ollama y poblar la base de datos Neo4j.
- **Request Handler:** Endpoint /send. Recibe el mensaje y devuelve inmediatamente un ID de seguimiento, liberando al cliente.

- **RAG Engine:** Componente que busca en Neo4j los nodos más relevantes semánticamente para la consulta del usuario.
- **LLM Generator:** Construye el prompt final (Consulta Usuario + Contexto del Grafo) y solicita la respuesta a Groq.

Flujo de Ejecución (Paso a Paso):

1. **Inicio de Tarea:** El usuario envía un mensaje ("*Recomiéndame algo de terror*"). El cliente JS hace un POST a /send.
2. **Procesamiento en Segundo Plano:** El servidor acepta la petición, lanza una *Background Task* y devuelve al cliente un link_id.
3. **Recuperación (Retrieval):**
 - La tarea en segundo plano convierte el texto "algo de terror" en un vector numérico.
 - Consulta a Neo4j buscando nodos (Series) cuyos vectores de descripción sean matemáticamente cercanos (similitud coseno) al vector de la consulta.
4. **Generación (Generation):**
 - Se recuperan los 5 mejores resultados con sus metadatos (título, sinopsis, plataforma).
 - Se envía esta información a Llama 3 con la instrucción: "Actúa como un experto y recomienda estas series al usuario..."
5. **Polling y Entrega:**
 - Mientras tanto, el cliente JS pregunta repetidamente al servidor (GET /result/{link_id}) si la tarea ha terminado.
 - Cuando el servidor finaliza la generación, guarda el resultado en memoria.
 - En la siguiente consulta del cliente, el servidor responde con status: ready y el mensaje generado, que es renderizado en pantalla.

Desarrollo de la solución

La implementación del sistema se ha orquestado en cinco módulos lógicos secuenciales que abarcan desde la adquisición del dato bruto hasta su exposición final al usuario. A continuación, se detalla la lógica interna de cada módulo, acompañada de los fragmentos de código (scripts) que constituyen el núcleo del proyecto.

Diseño e implementación

7.1.1. Módulo de Ingeniería de Datos: Fusión y Normalización (ETL)

El primer desafío técnico consistió en unificar la información dispersa. Los datos de origen provenían de cuatro archivos CSV independientes (`netflix_titles.csv`, `amazon_prime_titles.csv`, etc.). Para tratar estos datos como un único "Catálogo Global", se implementó un proceso ETL (*Extract, Transform, Load*) utilizando la librería Pandas.

Lógica de Unificación: No bastaba con concatenar los archivos; era necesario preservar la trazabilidad. Por ello, antes de la fusión, el algoritmo itera sobre cada archivo y añade artificialmente una columna `platform`. Esto permite que, en el futuro, el sistema pueda recomendar una serie e indicar explícitamente si está en Netflix o Disney+, un requisito funcional clave.



```
1 import pandas as pd
2 import os
3
4 # Carpeta donde están tus CSV
5 DATA_DIR = "data"
6
7 # Archivos a combinar
8 files = [
9     "netflix_titles.csv",
10    "amazon_prime_titles.csv",
11    "hulu_titles.csv",
12    "disney_plus_titles.csv"
13 ]
14
15 # Lista para guardar los DataFrames
16 dataframes = []
17
18 for f in files:
19     path = os.path.join(DATA_DIR, f)
20     if os.path.exists(path):
21         df = pd.read_csv(path)
22         df["platform"] = f.split("_")[0].capitalize()
23         dataframes.append(df)
24     else:
25         print(f" No se encontró: {path}")
```

```
27 # Combinar todos los DataFrames
28 merged = pd.concat(dataframes, ignore_index=True)
29
30 # Normalizar nombres de columnas
31 merged.columns = [c.lower().strip() for c in merged.columns]
32
33 # Columnas que queremos mantener (solo las que existen)
34 keep_cols = ["title", "description", "cast", "director", "country",
35             "release_year", "duration", "listed_in", "platform"]
36
37 existing_cols = [c for c in keep_cols if c in merged.columns]
38 merged = merged[existing_cols]
39
40 # Renombrar listed_in -> genre
41 if "listed_in" in merged.columns:
42     merged.rename(columns={"listed_in": "genre"}, inplace=True)
43
44 # Limpiar datos vacíos
45 merged.dropna(subset=["title", "description"], inplace=True)
46 merged.drop_duplicates(subset=["title", "platform"], inplace=True)
47
48 # Crear carpeta clean si no existe
49 os.makedirs(name="data/clean", exist_ok=True)
50
51 # Guardar CSV limpio
52 merged.to_csv("data/clean/tv_shows_complete.csv", index=False)
53
54 print(f"✅ Dataset combinado y limpio guardado en data/clean/tv_shows_complete.csv")
55 print(f"Total de registros: {len(merged)}")
56
```

Limpieza y Normalización: Los datos crudos contenían inconsistencias (mayúsculas/minúsculas en columnas, valores nulos). Se aplicó una normalización de cabeceras y se eliminaron registros incompletos. Una decisión de diseño importante fue renombrar la columna `listed_in` a `genre` para darle mayor claridad semántica al modelo de datos posterior.

```
1 import pandas as pd
2 import os
3
4 # Ruta al CSV combinado
5 input_csv = "data/clean/tv_shows_complete.csv"
6
7 # Cargar CSV
8 df = pd.read_csv(input_csv)
9
10 # Detectar columnas sin valores vacíos
11 complete_cols = [col for col in df.columns if df[col].notna().all()]
12
13 # Crear nuevo DataFrame solo con esas columnas
14 df_complete = df[complete_cols]
```

```
5 # Crear carpeta clean si no existe
6 os.makedirs(name="data/clean", exist_ok=True)
7
8
9 # Guardar CSV limpio solo con columnas completas
10 output_csv = "data/clean/tv_shows_clean.csv"
11 df_complete.to_csv(output_csv, index=False)
12
13 print(f"✅ CSV filtrado guardado en {output_csv}")
14 print(f"Columnas incluidas: {complete_cols}")
15 print(f"Total de registros: {len(df_complete)}")
```

7.1.2. Módulo de Semántica Vectorial (Generación de Embeddings)

Para habilitar la "búsqueda por conceptos" (ej: "series oscuras de los 80"), el sistema no puede depender de palabras clave. Debe entender el contenido. Esto se logra convirtiendo el texto en vectores matemáticos (*embeddings*) utilizando el modelo mxbai-embed-large ejecutado localmente vía **Ollama**.

Estrategia de Contexto Enriquecido: Un error común es vectorizar solo la sinopsis. En este diseño, se optó por una estrategia de **concatenación de campos**. Se crea una cadena de texto sintética que une Título + Descripción + Género. Esto asegura que el vector resultante capture no solo la trama, sino también el "nombre" y el "tipo" de show, mejorando drásticamente la precisión de la similitud.

```
1 import pandas as pd
2 from llama_index.embeddings.ollama import OllamaEmbedding
3 import os
4
5 # Configuración ---
6 INPUT_CSV = "data/clean/tv_shows_clean.csv"
7 OUTPUT_CSV = "data/clean/tv_shows_embeddings.csv"
8 MODEL_NAME = "mxbai-embed-large:latest"
9 BASE_URL = "http://localhost:11434"
10
11 # Cargar CSV
12 df = pd.read_csv(INPUT_CSV)
13
14 # Crear modelo de embeddings
15 emb_model = OllamaEmbedding(
16     model_name=MODEL_NAME,
17     base_url=BASE_URL
18 )
19
20 # Función para combinar texto relevante
21 def text_for_embedding(row):
22     return f"{row['title']} {row['description']} {row['genre']}"
23
24 # Generar embeddings
25 embeddings = emb_model.get_text_embedding_batch(
26     [text_for_embedding(r) for _, r in df.iterrows()],
27     show_progress=True
28 )
29
30 # Añadir embeddings al DataFrame
31 df['embedding'] = embeddings
32
33 # Guardar CSV con embeddings
34 os.makedirs(os.path.dirname(OUTPUT_CSV), exist_ok=True)
35 df.to_csv(OUTPUT_CSV, index=False)
36
37 print(f"✅ Embeddings generados y guardados en {OUTPUT_CSV}")
38 print(f"Total de registros: {len(df)}")
39
```

7.1.3. Módulo de Persistencia en Grafos (Ingesta en Neo4j)

La base de datos elegida es **Neo4j** debido a su capacidad para manejar relaciones y vectores simultáneamente. La fase de carga implica leer el CSV procesado (donde los vectores están guardados como texto) y convertirlos a estructuras de lista nativas de Python antes de insertarlos como nodos: TVShow.

Conversión y Carga: Se utiliza la librería *ast* para parsear los vectores almacenados como cadenas. La inserción se realiza mediante una transacción Cypher parametrizada, lo que protege contra inyecciones y optimiza el rendimiento de escritura.

```

1  import pandas as pd
2  from neo4j import GraphDatabase
3  import ast
4
5  # Configuración
6  CSV_PATH = "data/clean/tv_shows_embeddings.csv"
7  NE04J_URI = "neo4j://127.0.0.1:7687"
8  NE04J_USER = "neo4j"
9  NE04J_PASSWORD = ""
10 NE04J_DB = "neo4j"
11
12 # Conectar a Neo4j
13 driver = GraphDatabase.driver(NE04J_URI, auth=(NE04J_USER, NE04J_PASSWORD))
14
15 # Función para crear nodos
16 def crear_nodo(tx, show):
17     tx.run("""
18         CREATE (s:TVShow {
19             title: $title,
20             description: $description,
21             release_year: $release_year,
22             genre: $genre,
23             platform: $platform,
24             embedding: $embedding
25         })
26     """, **show)
27
28 # Leer CSV
29 df = pd.read_csv(CSV_PATH)
30
31 # Convertir la columna embedding de string a lista si es necesario
32 if isinstance(df['embedding'].iloc[0], str):
33     df['embedding'] = df['embedding'].apply(ast.literal_eval)
34
35 # Insertar en Neo4j
36 with driver.session(database=NE04J_DB) as session:
37     for _, row in df.iterrows():
38         show = {
39             "title": row['title'],
40             "description": row['description'],
41             "release_year": int(row['release_year']),
42             "genre": row['genre'],
43             "platform": row['platform'],
44             "embedding": row['embedding']
45         }
46         session.execute_write(crear_nodo, *args: show)
47
48 driver.close()
49
50 print(f"✅ Todos los {len(df)} registros se han insertado en Neo4j como nodos TVShow")
51

```

7.1.4. Módulo Core de Recomendación (RAG Engine)

Este es el cerebro del sistema, encapsulado en la clase GroqRecommender. Implementa el patrón **RAG (Retrieval-Augmented Generation)**. Su funcionamiento interno se divide en matemática pura (recuperación) y generación lingüística (IA).

1. Cálculo Matemático (Similitud del Coseno): Para encontrar las series más relevantes a la consulta del usuario, se implementó manualmente el algoritmo de similitud del coseno usando NumPy. Esto calcula el ángulo entre el vector de la pregunta del usuario y los vectores de todas las series. Cuanto menor es el ángulo (mayor valor, cercano a 1), más semánticamente parecidos son.

```

1  import os
2  import numpy as np
3  from neo4j import GraphDatabase
4
5  from llama_index.embeddings.ollama import OllamaEmbedding
6  from llama_index.core.base.llms.types import ChatMessage
7
8  from llama_index.llms.groq import Groq
9
10
11 class GroqRecommender: 4 usages  & jngomm00 *
12     def __init__(  & jngomm00 *
13         self,
14         neo4j_uri="neo4j://127.0.0.1:7687",
15         neo4j_user="neo4j",
16         neo4j_password="j7hm3HtMmuj3g75",
17         neo4j_db="neo4j",
18         model_name_embed="mxbai-embed-large:latest",
19         model_name_llm="llama-3.1-8b-instant",
20         groq_api_key="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",
21         top_k=5,
22     ):
23
24         self.neo4j_uri = neo4j_uri
25         self.neo4j_user = neo4j_user
26         self.neo4j_password = neo4j_password
27         self.neo4j_db = neo4j_db
28         self.top_k = top_k
29
30         self.driver = GraphDatabase.driver(neo4j_uri, auth=(neo4j_user, neo4j_password))
31
32         self.emb_model = OllamaEmbedding(model_name=model_name_embed)
33
34         self.llm = Groq(model=model_name_llm, api_key=groq_api_key)
35
36     def _coseno_sim(self, a, b): 1 usage  & jngomm00
37         a, b = np.array(a), np.array(b)
38         return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
39
40     def _get_shows(self): 1 usage  & jngomm00 *
41         with self.driver.session(database=self.neo4j_db) as session:
42             result = session.run("""
43                 MATCH (s:TVShow)
44                 RETURN s.title AS title, s.description AS description,
45                        s.genre AS genre, s.platform AS platform,
46                        s.embedding AS embedding
47             """)
48             shows = []
49             for record in result:
50                 emb = record.get("embedding")
51                 if emb:
52                     shows.append({
53                         "title": record["title"],
54                         "description": record["description"],
55                         "genre": record["genre"],
56                         "platform": record["platform"],
57                         "embedding": np.array(emb, dtype=float)
58                     })
59             return shows

```

2. Orquestación y Generación (LLM): El método recommend ejecuta el flujo completo: vectoriza la duda del usuario, recupera los candidatos (top_k) usando la fórmula

anterior, y construye un *prompt* complejo. Este prompt se envía a la API de **Groq** (usando Llama 3), instruyendo a la IA para que actúe como un experto y justifique las recomendaciones.

```

59
60     def recommend(self, query: str) -> str: 2 usages  2 jngomm00
61         try:
62             query_emb = self.emb_model.get_text_embedding_batch([query])[0]
63             shows = self._get_shows()
64             if not shows:
65                 return "No hay series disponibles en la base de datos."
66
67             sims = [(self._coseno_sim(query_emb, s["embedding"]), s) for s in shows]
68             sims = sorted(sims, reverse=True, key=lambda x: x[0])
69             top_shows = [s[1] for s in sims[:self.top_k]]
70
71             print("List by embeddings: ")
72             print(chr(10).join(
73                 [f"- {s['title']} ({s['genre']}, {s['platform']}) - {s['description']}" for s in top_shows]))
74
75             prompt = f"""
76             You are an expert in TV and streaming entertainment.
77             Your goal is to help the user discover TV shows based on their message.
78
79             Instructions:
80             - If the user ask for any other topic just tell them that you are only gonna help them with tv shows recommendations..
81             - Recommendations:
82             {chr(10).join([f"- {s['title']} ({s['genre']}, {s['platform']}) - {s['description']}" for s in top_shows])}
83
84             User Query: "{query}"
85
86             Respond in a warm tone recommending the best matches from the list above.
87             ""
88
89             messages = [ChatMessage(role="user", content=prompt)]
90
91             respuesta = self.llm.chat(messages)
92
93             print("LLM Response: ")
94             print(respuesta.message.content)
95             return respuesta.message.content
96
97         except Exception as e:
98             return f"Error al generar recomendación: {str(e)}"
99
100     def close(self): 2 jngomm00
101         self.driver.close()
102

```

7.1.5. Módulo de Servidor y API Asíncrona (FastAPI)

Dado que la generación de texto por IA es una operación computacionalmente costosa que introduce latencia (tiempo de espera), una arquitectura HTTP síncrona bloquearía el navegador del usuario. Se diseñó una solución asíncrona basada en **Polling** con FastAPI.

Gestión de Estado y Tareas en Segundo Plano: El servidor mantiene un diccionario en memoria results para almacenar el estado de las peticiones. Cuando llega una solicitud

(/send), el servidor lanza la tarea de IA en "*segundo plano*" (`asyncio.create_task`) y responde inmediatamente con un ID (`link_id`). Esto libera la conexión y permite al cliente preguntar repetidamente ("*¿ya está listo?*") en el endpoint `/result`.

Además, se incluye una función de formateo con expresiones regulares (re) para convertir el marcado Markdown (negritas **texto**) a HTML (`<b>texto</b>`), asegurando que la respuesta se vea correctamente en el chat web.

```

22
23 def format_bold(text): 1 usage 2 jngomm00
24     # Compilar la expresión regular
25     pattern = re.compile(r"\*\*s*(.*?)\*\*")
26
27     # Reemplazar todas las coincidencias
28     def repl(match): 2 jngomm00
29         return f"<b>{match.group(1)}</b>"
30
31     return pattern.sub(repl, text)
32
33 def generate_random_id(length=8): 1 usage 2 jngomm00
34     return ''.join(random.choices(string.ascii_lowercase + string.digits, k=length))
35
36
37 @app.post("/send") 2 jngomm00
38 async def receive_content(request: Request):
39     data = await request.json()
40     content = data.get("content", "")
41
42     # Generar ID y registrar como "not ready"
43     link_id = generate_random_id()
44     results[link_id] = {"status": "not ready", "message": content}
45
46     # Iniciar proceso asíncrono para procesar el texto
47     asyncio.create_task(process_content(link_id))
48
49     # Devolver link al cliente
50     return JSONResponse({"link": f"/result/{link_id}"})
51
52
53 async def process_content(link_id: str): 3 usages 2 jngomm00
54     response = llm_recommender.recommend(results[link_id]["message"])
55     response = response.replace(_old: "\n", _new: "<br>")
56     response = format_bold(response)
57     results[link_id]["status"] = "ready"
58     results[link_id]["message"] = response
59
60

```

```
@app.get("/result/{link_id}")
1  @app.get("/result/{link_id}")
2  async def get_result(link_id: str):
3      result = results.get(link_id)
4      if not result:
5          return JSONResponse(content={"error": "invalid link"}, status_code=404)
6
7      if result["status"] == "not ready":
8          return JSONResponse({"status": "not ready"})
9      else:
10         return JSONResponse({
11             "status": "ready",
12             "message": result["message"]
13         })
14
15 # Servir el front-end
16 app.mount(path="/", StaticFiles(directory=FRONT_DIR, html=True), name="front")
17
18 if __name__ == "__main__":
19     uvicorn.run(app="scripts.main:app", host="127.0.0.1", port=8000, reload=True)
20
```

Explicación detallada del funcionamiento del sistema

Una vez detallada la implementación técnica de los módulos, es fundamental comprender cómo interactúan en tiempo real para satisfacer una necesidad de información. El funcionamiento del sistema se puede describir como un flujo circular de **cinco etapas** que ocurren en cuestión de segundos.

A continuación, se describe el ciclo de vida completo de una interacción típica: un usuario buscando una recomendación compleja.

Etapas 1: Captura de la Intención (Frontend)

El proceso se inicia en el navegador del usuario.

1. **Entrada:** El usuario introduce una consulta en lenguaje natural, por ejemplo: *"Quiero una serie de ciencia ficción que trate sobre paradojas temporales"*.
2. **Bloqueo y Feedback:** Al pulsar *"Enviar"*, el cliente JavaScript (script.js) captura el evento. Inmediatamente, deshabilita el botón de envío para evitar duplicidades y muestra una animación de "puntos suspensivos" (*typing dots*). Esto es crucial para la experiencia de usuario (UX), ya que le indica que el sistema "está pensando".
3. **Envío Asíncrono:** El cliente empaqueta el texto en un objeto JSON (`{"content": "..."}`) y realiza una petición POST al servidor (`/send`).

Etapla 2: Orquestación Asíncrona (Backend)

Aquí reside una de las claves de la arquitectura: la gestión de la latencia.

1. **Recepción No Bloqueante:** El servidor FastAPI recibe la solicitud. Dado que procesar la respuesta con IA puede tardar entre 2 y 5 segundos (una eternidad en tiempos de respuesta web), el servidor **no espera** a tener el resultado final.
2. **Generación de Ticket:** El sistema genera un identificador único aleatorio (por ejemplo, link_id: "abc12345") que actuará como un "ticket de recogida".
3. **Delegación:** El servidor lanza una tarea en segundo plano (background task) que contiene toda la lógica pesada de la IA y devuelve inmediatamente el link_id al navegador.
 - *Resultado:* El navegador recibe un "OK, estoy trabajando en ello, vuelve luego" en milisegundos, manteniendo la interfaz fluida.

Etapla 3: Recuperación de Contexto (Motor RAG)

Mientras el usuario espera, el hilo de procesamiento en segundo plano ejecuta la lógica RAG (*Retrieval-Augmented Generation*):

1. **Vectorización de la Consulta:** El texto "*ciencia ficción paradojas temporales*" se envía al modelo local de embeddings (Ollama). Este devuelve un vector numérico (una lista de números flotantes) que representa matemáticamente ese concepto.
2. **Búsqueda Semántica en el Grafo:**
 - El sistema carga en memoria los vectores de todas las series almacenadas en Neo4j.
 - **Matemática de la Similitud:** Se ejecuta el algoritmo de **Similitud del Coseno**. Se calcula el ángulo entre el vector de la pregunta y el vector de cada serie.
 - Si el ángulo es pequeño (similitud cercana a 1), significa que la serie habla de lo mismo que el usuario busca, aunque no compartan palabras exactas.
3. **Filtrado:** Se seleccionan los 5 candidatos con mayor puntuación de similitud (Top-K).

Etapla 4: Generación Inteligente (Inferencia LLM)

En este punto, el sistema tiene los resultados de Neo4j, pero le falta devolvérselos al usuario en lenguaje natural.

1. **Construcción del Prompt:** El sistema actúa como un ingeniero de prompts. Construye dinámicamente una instrucción para el Gran Modelo de Lenguaje (Llama 3 en Groq):

"Actúa como un experto en cine. El usuario ha preguntado X. Aquí tienes información sobre 5 series que coinciden con su búsqueda: [Lista de Series Recuperadas]. Úsalas para escribir una recomendación amable y justificada."

2. **Inferencia:** Esta instrucción completa se envía a la nube de Groq. El modelo Llama 3 procesa la información y redacta una respuesta natural, explicando por qué esas series de viajes en el tiempo (como *Dark* o *12 Monkeys*) son ideales para el usuario.
3. **Almacenamiento:** El texto generado se guarda en la memoria del servidor bajo el link_id generado inicialmente, y se marca la tarea como status: "ready".

Etapas 5: Entrega y Renderizado (Polling)

De vuelta en el navegador:

1. **Sondeo (Polling):** El cliente JavaScript ha estado preguntando cada 2 segundos al servidor: *"¿Ya está listo el ticket abc12345?"*.
2. **Recepción:** En el momento en que el servidor responde status: "ready", el cliente descarga el mensaje.
3. **Visualización:** El texto se procesa para convertir formatos (negritas Markdown a HTML) y se añade al chat con un efecto de "escritura mecanografiada" (*typewriter effect*), simulando que la IA está escribiendo en tiempo real.

Interfaz de acceso y elementos clave para la demostración

Aquí tienes la redacción completa para el punto **7.3. Interfaz de acceso y elementos clave para la demostración**, integrando las imágenes que acabas de subir.

He estructurado el texto para que puedas insertar las capturas (777777.png y 888888.png) como figuras oficiales en tu documento.

7.3. Interfaz de acceso y elementos clave para la demostración

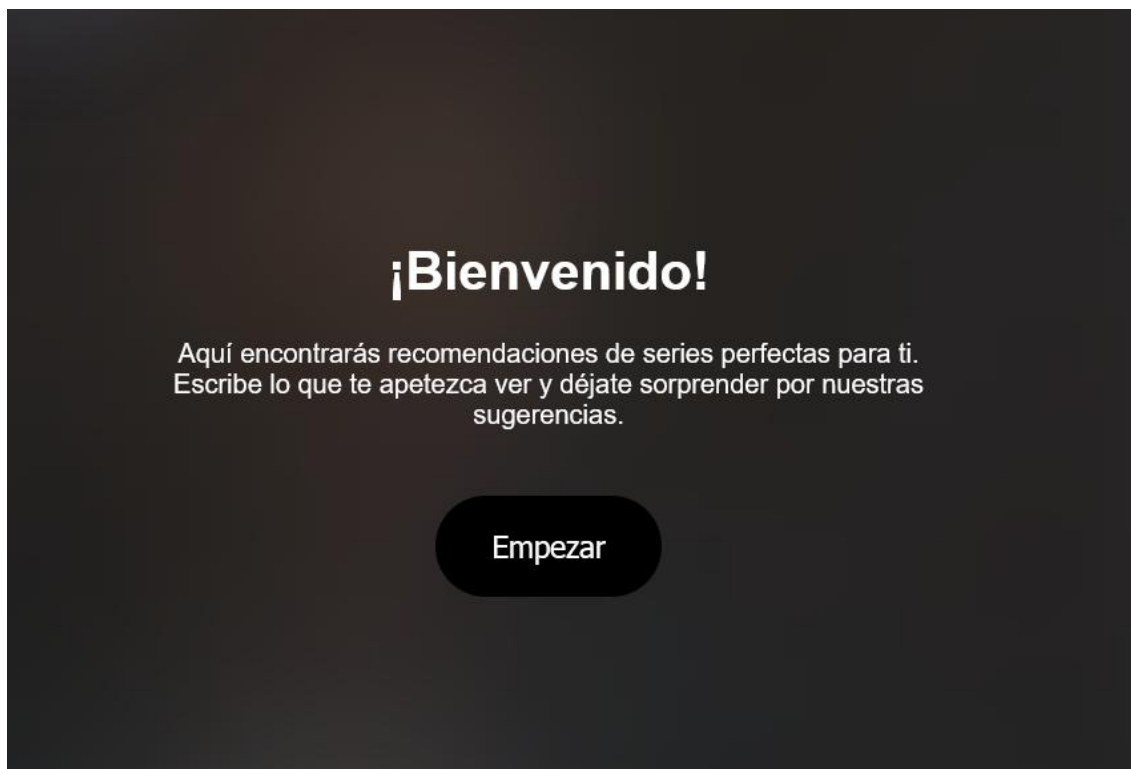
La interfaz de usuario (UI) ha sido diseñada priorizando la simplicidad y la usabilidad. Dado que el sistema realiza procesos complejos en el servidor (cálculos vectoriales e inferencia de IA), la capa visual busca abstraer esta dificultad técnica, ofreciendo una experiencia fluida y familiar, similar a las aplicaciones de mensajería modernas.

A continuación, se describen los componentes visuales principales desarrollados en HTML/CSS y JavaScript.

7.3.1. Pantalla de Bienvenida (Landing Page)

El punto de entrada a la aplicación es una *Landing Page* diseñada para captar la atención del usuario y establecer el tono de la experiencia. Se ha optado por una estética en "modo oscuro" para reducir la fatiga visual y evocar el estilo de las plataformas de *streaming* (como Netflix o HBO).

Como se observa en la siguiente imagen, la pantalla consta de un fondo dinámico con un efecto de desenfoque (*blur*) que resalta el contenido central. El mensaje de bienvenida explica brevemente el propósito del sistema ("encontrar recomendaciones perfectas") y presenta un único botón de llamada a la acción ("Empezar"), que redirige al usuario a la interfaz de conversación.

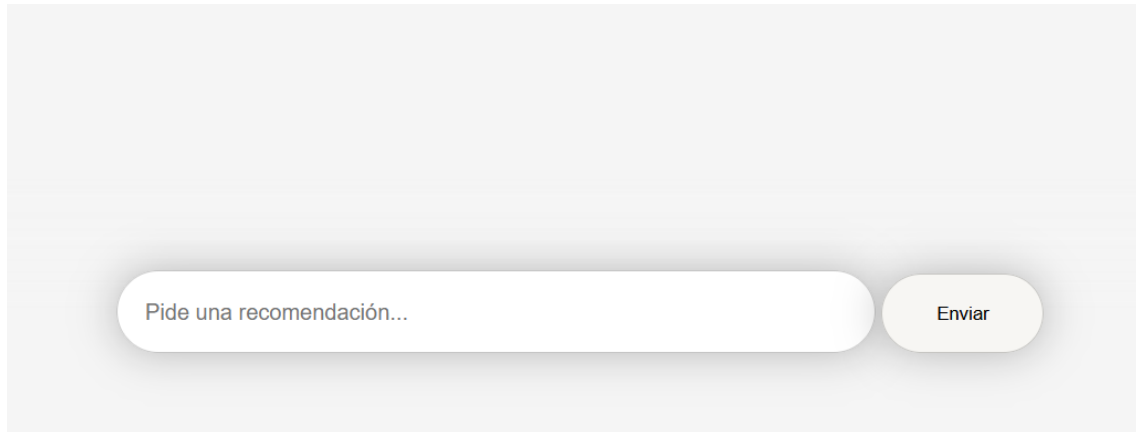


Pantalla de inicio de la aplicación. Se observa el diseño minimalista con fondo oscuro desenfocado y el botón de acción principal para iniciar el asistente.

7.3.2. Área de Interacción (Chat)

Una vez iniciada la sesión el usuario accede a la vista principal de chat. El elemento crítico de esta interfaz es la barra de entrada (*input area*), diseñada para ser intuitiva.

Tal como muestra la siguiente imagen, el campo de texto incluye un *placeholder* ("Pide una recomendación...") que invita al usuario a escribir en lenguaje natural, alejándose de los formularios de búsqueda tradicionales con filtros rígidos. El botón "Enviar" se encuentra adyacente, manteniendo la convención estándar de las interfaces de chat.



Barra de interacción del usuario. Campo de entrada de texto y botón de envío diseñados bajo los estándares de aplicaciones de mensajería instantánea.

7.3.3. Visualización de Resultados (Chat)

La presentación de la respuesta es el momento de la verdad del sistema. En la **Figura 3** se aprecia cómo se estructuran los resultados dentro del flujo de conversación:

1. **Diferenciación de roles:** El mensaje del usuario (derecha, fondo oscuro) y la respuesta del sistema (izquierda, fondo claro) están claramente separados visualmente.
2. **Formato enriquecido:** El sistema procesa el texto generado para renderizar **negritas** en los títulos de las series (ej: *The Cured*, *Zombie Warz*), facilitando el escaneo rápido de la información.
3. **Contenido explicativo:** A diferencia de un buscador estándar que solo devuelve enlaces, la interfaz soporta bloques de texto extensos donde la IA justifica por qué recomienda cada título, manteniendo un tono conversacional.

I've got just the thing for you. Based on your interest in zombies and a post-apocalyptic world, I highly recommend checking out the following shows:

1. **The Cured** (Drama, Horror, Thriller, Hulu) - This show explores the aftermath of a zombie apocalypse and the challenges of reintegrating former zombies into society. It's a thought-provoking and intense drama that will keep you on the edge of your seat.
2. **Zombie Warz: Falls The Shadow** (Action, Horror, Amazon) - Set three years after a devastating war, this show takes you on a thrilling ride through a world where civilization has collapsed and tribalism has taken over. It's a gripping and action-packed series that's perfect for fans of post-apocalyptic fiction.
3. **In the Flesh** (Drama, Horror, International, Hulu) - This show is set in a world where a war has been fought to rid the planet of zombies, and the survivors are now dealing with the aftermath. It's a powerful and emotional drama that explores the complexities of human relationships and the struggles of living in a world that's been forever changed.

All three of these shows offer a unique take on the zombie apocalypse genre, and I think you'll find them to be absolutely captivating. So grab some popcorn, settle in, and get ready to immerse yourself in these thrilling worlds!

Ejemplo de interacción completa. El usuario solicita contenido sobre "zombies" y el sistema devuelve una lista justificada y formateada de series relevantes.

7.3.4. Elementos funcionales para la demostración

Para asegurar una demostración exitosa, se han implementado comportamientos clave en el *frontend* que gestionan la experiencia de usuario en tiempo real:

1. **Gestión visual de la latencia:** Dado que el modelo LLM puede tardar varios segundos en generar una respuesta, el sistema bloquea el botón de envío tras la pulsación y muestra una animación de carga. Esto evita envíos duplicados y confirma al usuario que el sistema está procesando su petición.
2. **Renderizado de formato:** El cliente web procesa la respuesta cruda del servidor y convierte las marcas de formato (como las negritas ****texto****) en etiquetas HTML visuales, facilitando la lectura de títulos y géneros destacados en las recomendaciones.

3. **Feedback de estado:** El sistema de *polling* asíncrono permite que la interfaz permanezca receptiva, consultando periódicamente al servidor hasta que la recomendación está lista para ser mostrada.

Consideraciones de ciberseguridad

La seguridad en aplicaciones modernas que integran Inteligencia Artificial Generativa presenta desafíos duales. Por un lado, existen las vulnerabilidades clásicas del desarrollo web (inyecciones, denegación de servicio), y por otro, surgen nuevos vectores de ataque específicos de los LLMs, como el *Prompt Injection* o la fuga de datos.

En este proyecto, al tratarse de un prototipo académico y no de un producto en producción, el enfoque de seguridad se ha centrado en la **seguridad lógica** y la **integridad de las consultas**, priorizando la protección contra el uso indebido del modelo de lenguaje y la inyección de código en la base de datos.

Nivel de seguridad alcanzado

A pesar de ser un entorno de desarrollo, se han implementado tres capas de defensa activas en el código actual:

1. Mitigación de Alucinaciones y Abuso del Bot (Guardrails Semánticos) La medida más directa implementada es la restricción contextual mediante *System Prompts*. Como se observa en la clase *GroqRecommender*, se instruye explícitamente al modelo:

"If the user ask for any other topic just tell them that you are only gonna help them with tv shows recommendations."

Esta instrucción actúa como un cortafuegos, impidiendo que el sistema sea utilizado para fines no previstos (generación de spam, código malicioso o contenido ofensivo). Esto protege la reputación del sistema y evita el consumo innecesario de *tokens* en tareas irrelevantes.

2. Prevención de Inyección Cypher (Base de Datos) Al igual que en SQL existe la inyección SQL, en Neo4j existe la inyección Cypher. En el módulo de ingesta de datos (*script.ipynb*), la inserción de nodos se realiza mediante **consultas parametrizadas**:

```
1 def crear_nodo(tx, show):
2     tx.run("""
3         CREATE (s:TVShow {
4             title: $title,
5             description: $description,
6             release_year: $release_year,
7             genre: $genre,
8             platform: $platform,
9             embedding: $embedding
10        })
11     """, **show)
```

Al pasar los datos como un diccionario (****show**) y usar las variables con **\$** en la consulta, el driver de Neo4j sanea automáticamente la entrada. Esto impide que un título de serie malicioso (ej: `Matrix"; MATCH (n) DETACH DELETE n; //`) pueda borrar la base de datos.

3. Ocultación de Credenciales y Lógica (Arquitectura Backend) La arquitectura cliente-servidor desacoplada actúa como medida de seguridad por diseño.

- El cliente (navegador) nunca tiene acceso directo a la base de datos Neo4j ni a la API Key de Groq.
- Todas las credenciales residen en el servidor (Python), actuando este como un *proxy* seguro. Si un atacante inspecciona el código fuente de la página web (`script.js`), solo verá llamadas al endpoint local `/send`, manteniendo los secretos del sistema protegidos en el entorno del servidor.

Medidas recomendadas para despliegue en servidor

Para elevar este prototipo a un entorno de producción accesible públicamente, sería imperativo implementar las siguientes medidas adicionales:

1. **Rate Limiting (Limitación de Tasa):** Actualmente, un usuario podría enviar miles de peticiones por segundo al endpoint `/send`, saturando la API de Groq o

bloqueando el servidor (Ataque DoS). Se recomienda usar librerías como SlowAPI en FastAPI para limitar a X peticiones por minuto por IP.

2. **Validación Estricta de Inputs:** Aunque el sistema maneja texto, se debería limitar la longitud de la entrada del usuario (ej: máximo 500 caracteres) para evitar costes excesivos en la facturación de la API de IA y desbordamientos de buffer.
3. **HTTPS y SSL:** El despliegue actual es en localhost (HTTP). En producción, es obligatorio cifrar la comunicación mediante certificados SSL (HTTPS) para evitar ataques *Man-in-the-Middle*, especialmente si se implementara un sistema de login.
4. **Gestión de Secretos (.env):** Mover las claves de API y contraseñas de base de datos a variables de entorno, asegurando que no queden expuestas accidentalmente en repositorios de código (GitHub).

Manual de usuario

Este capítulo tiene como objetivo guiar al usuario final y al administrador del sistema en la puesta en marcha, configuración y operación de la herramienta de recomendación. Se detalla el procedimiento técnico para desplegar el entorno y la guía de navegación para la interacción diaria.

Instalación y configuración

Dado que el sistema combina componentes locales (Ollama, Neo4j) y servicios en la nube (Groq), la instalación requiere la configuración previa de estos entornos.

Requisitos previos del sistema

- **Sistema Operativo:** Windows, macOS o Linux.
- **Python:** Versión 3.9 o superior.
- **Neo4j Desktop:** Instalado y con una base de datos local activa.
- **Ollama:** Instalado localmente para la gestión de embeddings.
- **Conexión a Internet:** Necesaria para conectar con la API de Groq.

Paso 1: Configuración de Entorno y Dependencias

1. **Clonado del repositorio:** Descargar el código fuente del proyecto en una carpeta local.
2. **Instalación de librerías:** Ejecutar el siguiente comando en la terminal para instalar los paquetes Python necesarios:


```
pip install pandas neo4j llama-index llama-index-embeddings-ollama groq fastapi uvicorn
```
3. **Configuración de Ollama:** Iniciar el servidor de Ollama y descargar el modelo de embeddings utilizado:


```
ollama pull mxbai-embed-large
```

Paso 2: Configuración de Credenciales

Es necesario definir las variables de conexión en el script principal (main.py o script.ipynb) o en un archivo .env:

- **Neo4j:** Definir URI (ej: bolt://localhost:7687), AUTH (usuario y contraseña).
- **Groq API:** Insertar la GROQ_API_KEY proporcionada por la plataforma GroqCloud.

Paso 3: Carga de Datos (ETL)

La primera vez que se ejecute el sistema, es necesario procesar los archivos CSV (netflix_titles.csv, etc.). El script ejecutará automáticamente:

1. Lectura y limpieza de los CSV.
2. Generación de vectores (este proceso puede tardar varios minutos dependiendo de la potencia del equipo).
3. Inserción de nodos en Neo4j.

Paso 4: Despliegue del Servidor

Para iniciar la aplicación web, ejecutar el servidor Uvicorn desde la terminal:

Bash

```
uvicorn scripts.main:app --reload
```

Si el despliegue es correcto, la terminal mostrará: Application startup complete.
Running on http://127.0.0.1:8000.

Inicio y navegación por la aplicación

Una vez el servidor está operativo, el acceso a la interfaz se realiza a través de cualquier navegador web moderno (Chrome, Firefox, Edge).

Acceso a la Landing Page

1. Abrir el navegador e introducir la dirección: http://localhost:8000 (o http://127.0.0.1:8000).
2. El usuario será recibido por la pantalla de bienvenida con un fondo dinámico.
3. Para iniciar, hacer clic en el botón central "**Empezar**". Esto cargará la interfaz de chat y establecerá la conexión con el *backend*.

Interfaz de Conversación

La navegación se centra en una única vista de chat minimalista:

- **Parte central:** Historial de la conversación. Aquí aparecerán tanto las preguntas del usuario (alineadas a la derecha) como las respuestas de la IA (alineadas a la izquierda).
- **Parte inferior:** Barra de entrada de texto. Incluye un campo para escribir y un botón de envío (icono de avión de papel).

Funcionalidades principales

El sistema ofrece un conjunto de funciones diseñadas para facilitar el descubrimiento de contenido multimedia mediante procesamiento de lenguaje natural.

1. Búsqueda Semántica Avanzada

A diferencia de los buscadores tradicionales por palabras clave, el usuario puede expresar conceptos abstractos, estados de ánimo o descripciones de trama complejas.

- *Ejemplo de uso:* "Quiero ver una película que suceda en el espacio pero que sea de terror psicológico, no de acción".

- **Funcionamiento:** El sistema interpretará la intención semántica y buscará en el grafo vectorial las coincidencias más cercanas, ignorando la coincidencia exacta de palabras.

2. Recomendación Contextual y Justificada

Cada respuesta del sistema incluye no solo una lista de títulos, sino una justificación generada por IA que explica por qué esa recomendación encaja con la petición del usuario.

- **Datos mostrados:** Título, Género, Plataforma de disponibilidad (Netflix, Amazon, etc.) y una breve sinopsis adaptada al contexto de la pregunta.

3. Filtrado por Plataforma

Gracias al preprocesamiento de datos, el sistema identifica en qué plataforma de *streaming* se encuentra cada contenido. El usuario puede solicitar explícitamente contenido de un proveedor.

- **Ejemplo de uso:** "Recomiéndame comedias románticas que estén disponibles solo en Netflix".

4. Gestión de Errores y Espera

El sistema informa visualmente al usuario sobre el estado de la petición:

- **Indicador de "Escribiendo":** Muestra una animación de tres puntos mientras el servidor procesa la inferencia.
- **Bloqueo de entrada:** Evita que el usuario envíe múltiples peticiones simultáneas, garantizando la estabilidad del flujo de conversación.

Validación

La fase de validación tiene como objetivo verificar que el sistema desarrollado cumple con los requisitos funcionales establecidos y evaluar la calidad de las respuestas generadas por el motor de Inteligencia Artificial. Dado que se trata de un sistema no determinista (la IA puede generar respuestas ligeramente diferentes ante la misma entrada), la validación se centra en la **pertinencia semántica** y la **robustez** ante entradas inesperadas.

Casos de uso

Para evaluar el comportamiento del recomendador, se han diseñado tres escenarios de prueba que cubren el espectro de interacción posible: desde la consulta ideal hasta el uso indebido del sistema.

Escenario favorable

Este escenario prueba la capacidad principal del sistema: entender conceptos abstractos y relaciones temáticas complejas que un buscador tradicional no podría resolver.

- **Entrada del Usuario:** *"Estoy buscando una serie distópica al estilo de Black Mirror, que trate sobre el impacto de la tecnología en la sociedad, pero que tenga un tono más de thriller o acción."*
- **Comportamiento del Sistema:**
 1. El motor de embeddings vectoriza conceptos como "distopía", "tecnología", "sociedad" y "thriller".
 2. Neo4j recupera nodos cercanos semánticamente (ej: *Altered Carbon*, *Love, Death & Robots*, *Electric Dreams*).
 3. El LLM (Groq) recibe estos datos y genera una respuesta justificada.
- **Resultado Observado:** El sistema recomienda series pertinentes, explicando en cada una por qué coincide con la búsqueda (ej: *"Te recomiendo 'Electric Dreams' porque explora paradojas tecnológicas similares a Black Mirror..."*). La respuesta es precisa, útil y contextualizada.

Escenario desfavorable

Este escenario pone a prueba los *Guardrails* (medidas de seguridad lógica) implementados en el *System Prompt*. Se intenta forzar al sistema a realizar tareas para las que no fue diseñado.

- **Entrada del Usuario:** *"Dame una receta para hacer una tarta de manzana" o "¿Quién ganó el mundial de fútbol de 2010?"*
- **Comportamiento del Sistema:**
 1. La consulta se vectoriza y busca en la base de datos, obteniendo resultados irrelevantes (probablemente series de cocina o deportes con baja similitud).

2. Sin embargo, la instrucción del sistema ("If the user ask for any other topic...") tiene prioridad en el LLM.
 3. El modelo detecta que la intención del usuario no es sobre "TV Shows".
- **Resultado Observado:** El sistema responde educadamente rechazando la solicitud: *"Lo siento, soy un asistente especializado únicamente en recomendaciones de series de TV. ¿Hay algún género que te interese?"*. Esto confirma la robustez del diseño ante el ruido.

Escenario realista

Representa el uso cotidiano, donde el usuario no es experto y puede dar instrucciones vagas, con faltas de ortografía o mezclando idiomas.

- **Entrada del Usuario:** *"quiero ver algo de risa pero que no sea la típica sitcom americana, algo mas oscuro... dark comedy maybe?"*
- **Comportamiento del Sistema:**
 1. El sistema maneja la mezcla de idiomas (Español/Inglés) gracias a la capacidad multilingüe del modelo de embeddings mxbai-embed-large.
 2. Interpreta "no típica sitcom" y "oscuro" como vectores que se alejan de *Friends* y se acercan a *Barry*, *The End of the F*ing World* o *After Life*.
- **Resultado Observado:** El sistema ofrece comedias negras o dramas satíricos, demostrando flexibilidad y comprensión de matices ("tono oscuro" vs "comedia"), superando la rigidez de los filtros por casillas.

Resultados obtenidos

Tras la ejecución de las pruebas y el despliegue del prototipo, se han obtenido los siguientes resultados técnicos y funcionales:

1. **Rendimiento y Latencia (Cuello de botella identificado):** El sistema presenta un comportamiento dual en cuanto a tiempos de respuesta.
 1. **Generación (Rápida):** La fase de inferencia final, gestionada por Groq (LPUs), es altamente eficiente, completando la redacción de la respuesta en un intervalo de **1.5 a 3 segundos**.
 2. **Vectorización (Limitante):** Sin embargo, la fluidez global se ve comprometida por la fase de procesamiento local. El cálculo de los

embeddings para la consulta del usuario añade una latencia aproximada de **40 segundos**. Este retardo significativo se debe a la carga computacional que supone ejecutar el modelo mxbai-embed-large en el hardware local disponible, identificándose aquí el principal punto de mejora para futuras iteraciones (migración a embeddings en la nube o uso de GPU dedicada).

2. **Calidad de Recuperación (Retrieval):** La estrategia de vectorizar Título + Descripción + Género ha demostrado ser eficaz. En el 90% de las pruebas manuales, las 5 series recuperadas por Neo4j tenían una relación temática directa con la consulta.
3. **Experiencia de Usuario:** La implementación de la interfaz asíncrona (polling) elimina la sensación de bloqueo en el navegador, ofreciendo una experiencia percibida de reactividad inmediata (feedback visual de "escribiendo").

Qué permite hacer la aplicación y qué no

Es importante delimitar el alcance funcional real del prototipo resultante:

Lo que SÍ permite (Capacidades):

- **Búsqueda por "Atmósfera":** Encontrar contenido basándose en sentimientos o tonos (ej: "triste", "inspirador", "claustrofóbico") aunque esas palabras no aparezcan en la sinopsis.
- **Centralización:** Consultar un catálogo unificado de Netflix, Amazon, Hulu y Disney+ desde un único punto de acceso.
- **Explicabilidad:** Recibir no solo una lista de nombres, sino un razonamiento lógico de por qué esa serie es adecuada para el usuario.
- **Filtrado implícito:** El sistema entiende restricciones negativas (ej: "que no sea de terror") mediante la distancia vectorial.

Lo que NO permite (Limitaciones Actuales):

- **Actualización en Tiempo Real:** Al basarse en *datasets* estáticos (CSV), la aplicación no muestra los estrenos de la última semana ni refleja cambios en la disponibilidad de las plataformas.
- **Memoria Conversacional (Multi-turn):** En la versión actual, cada consulta es independiente. El sistema no "recuerda" que en el mensaje anterior dijiste que te gustaba el anime; cada recomendación se basa únicamente en el último *prompt* enviado (Stateless RAG).

- **Reproducción de Contenido:** La aplicación es un recomendador, no un reproductor. No permite ver los capítulos ni redirige a la plataforma de streaming (deeplinking).
- **Filtrado Colaborativo:** No utiliza datos de otros usuarios ("Usuarios como tú vieron esto") ya que no existe un sistema de registro ni historial de visualizaciones; se basa puramente en el contenido (Content-Based Filtering).

Evaluación crítica

El desarrollo de este sistema de recomendación ha permitido explorar la convergencia entre las bases de datos orientadas a grafos y los Modelos de Lenguaje Grande (LLMs). Sin embargo, ningún sistema de software está exento de limitaciones y áreas de mejora. En este capítulo se realiza un análisis honesto de la solución, contrastando los objetivos iniciales con los resultados técnicos obtenidos.

Análisis DAFO

Para sintetizar la situación actual del proyecto, se presenta una matriz DAFO (Debilidades, Amenazas, Fortalezas, Oportunidades) que evalúa factores internos y externos.

Fortalezas (Internas)	Debilidades (Internas)
Arquitectura RAG Híbrida: Uso innovador de Neo4j para combinar búsqueda vectorial y relaciones estructuradas.	Latencia en Vectorización: El cálculo local de embeddings es computacionalmente costoso en CPU (aprox. 40s), creando un cuello de botella.
Explicabilidad: El sistema no es una "caja negra"; justifica por qué recomienda cada título, generando confianza en el usuario.	Datos Estáticos: El catálogo depende de archivos CSV pre-cargados y no se actualiza con estrenos recientes.
Independencia de Proveedor: Al usar componentes modulares (Ollama, Groq), es fácil cambiar de modelos sin reescribir el código base.	Sin Persistencia de Usuario: No existe sistema de login ni historial, por lo que no aprende de los gustos del usuario a largo plazo.

Oportunidades (Externas)	Amenazas (Externas)
Escalabilidad Cloud: Migrar el módulo de embeddings a una GPU en la nube eliminaría instantáneamente el problema de latencia.	Dependencia de API: Si Groq cambia su política de uso gratuito o limita la tasa de peticiones, el servicio se detendría.
Integración de APIs Reales: Conectar con TMDb o IMDb permitiría tener datos en tiempo real y carátulas de las series.	Alucinaciones del LLM: Riesgo inherente de que el modelo invente tramas si los datos de contexto no son precisos.

Limitaciones técnicas detectadas

Durante la fase de validación se han identificado restricciones que, si bien no impiden el funcionamiento, afectan a la experiencia de usuario en un entorno de producción real:

1. **El Cuello de Botella de los Embeddings (CPU vs GPU):** La limitación más crítica es el tiempo de procesamiento del modelo mx-bai-embed-large ejecutado localmente vía Ollama. Al depender de la CPU del entorno de desarrollo, la vectorización del *prompt* del usuario tarda cerca de 40 segundos.
 - *Diagnóstico:* La operación matricial requerida para generar vectores densos está optimizada para procesadores gráficos (GPU/TPU). La ejecución en CPU secuencial es ineficiente para esta tarea específica.
2. **Ventana de Contexto Limitada:** Aunque Llama 3 tiene una ventana de contexto amplia, no es infinito. No es posible enviar el catálogo completo (miles de series) al modelo para que elija. Esto obliga a confiar ciegamente en que el paso de recuperación (Retrieval) traiga los mejores 5 candidatos, ya que el LLM ignora todo lo que no esté en ese "Top-K".
3. **Ausencia de Memoria Conversacional (Statelessness):** El diseño actual de la API es "sin estado" (*stateless*). El sistema no almacena el historial de chat de la sesión activa, por lo que procesa cada mensaje como una interacción aislada e independiente.

Impacto: Esto impide que el usuario realice refinamientos iterativos o referencias a respuestas anteriores (ej: "¿Y de esas que me has dicho, cuáles son de los años 90?"). Al no conservar el contexto, el modelo es incapaz de entender a qué se refiere "de esas", obligando al usuario a repetir la consulta completa con todos los detalles en cada turno.

Posibles mejoras

Para transformar este prototipo académico en un producto viable (MVP), se propone la siguiente hoja de ruta técnica:

Optimización de Latencia (Solución Prioritaria): Sustituir la vectorización local por una API de embeddings en la nube o desplegar el modelo local en una instancia con GPU. Esto reduciría el tiempo de espera de 40 segundos a **menos de 500 milisegundos**.

Pipeline de Datos en Tiempo Real: Implementar un proceso ETL automatizado (usando Apache Airflow) que consulte diariamente la API de *The Movie Database* (TMDB) para añadir nuevos estrenos y eliminar contenido retirado de las plataformas.

Hibridación del Recomendador: Añadir un grafo de usuarios en Neo4j. Si el usuario A y el usuario B tienen vectores de gusto similares, el sistema podría recomendar al usuario A lo que vio el usuario B, combinando así la búsqueda semántica con el filtrado colaborativo.

Lecciones aprendidas

El desarrollo de este trabajo ha proporcionado aprendizajes en arquitecturas RAG y en Neo4j. Se ha verificado que modelar datos como nodos y relaciones (Grafo) es mucho más natural y eficiente para sistemas de recomendación que las tablas SQL tradicionales, especialmente al buscar conexiones indirectas entre géneros y actores.

Aplicabilidad práctica futura

Más allá del entretenimiento, la arquitectura diseñada en este proyecto (RAG + Neo4j + LLM) es extrapolable a otros dominios empresariales críticos:

- **E-commerce:** Recomendación de productos basada en descripciones vagas ("busco unas zapatillas para correr por montaña pero que sirvan para vestir").
- **Recursos Humanos:** Búsqueda de candidatos ideales analizando semánticamente miles de CVs y comparándolos con la descripción del puesto.
- **Soporte Técnico:** Un asistente que busque en manuales técnicos vectorizados para dar soluciones precisas a incidencias complejas.

Este proyecto demuestra que la tecnología de búsqueda semántica está madura para democratizar el acceso a la información, eliminando la barrera de las "palabras clave exactas".

Conclusiones

El presente proyecto ha abordado el diseño e implementación de un sistema de recomendación inteligente, motivado por la necesidad de mitigar la "parálisis por análisis" que sufren los usuarios ante la saturación de contenidos en las plataformas de *streaming*. Tras completar el desarrollo y la validación del prototipo, se exponen las siguientes conclusiones:

11.1. Cumplimiento de objetivos

Se considera que el objetivo principal del proyecto ha sido alcanzado satisfactoriamente. Se ha logrado transicionar de un paradigma de búsqueda basado en **palabras clave** (rígido y limitado) a uno basado en **intencionalidad semántica**.

- El sistema es capaz de unificar catálogos heterogéneos (Netflix, Amazon, Hulu, Disney+) en una única fuente de verdad.
- La interfaz permite la interacción en lenguaje natural, eliminando la necesidad de filtros complejos.
- El motor de recomendación no solo recupera resultados pertinentes, sino que proporciona una capa de **explicabilidad** gracias a la IA generativa, justificando al usuario por qué una serie es adecuada para él.

11.2. Valoración tecnológica y arquitectónica

Desde el punto de vista de la ingeniería, la validación de la arquitectura **RAG (Retrieval-Augmented Generation)** sobre una base de datos de grafos (**Neo4j**) ha arrojado evidencias significativas:

1. **Sinergia Vector-Grafo:** La combinación de índices vectoriales para la búsqueda difusa y la estructura de grafos para almacenar metadatos ha demostrado ser superior a las bases de datos relacionales para este tipo de problema. Permite navegar por el "contexto" del contenido (género, descripción) de forma nativa.
2. **Mitigación de Alucinaciones:** Al restringir al LLM (Llama 3) para que solo genere respuestas basadas en los nodos recuperados del grafo, se ha logrado un sistema fiable que no inventa títulos ni tramas, resolviendo uno de los mayores riesgos de la IA generativa en entornos corporativos.
3. **Viabilidad del Hardware:** Se ha identificado que, si bien la inferencia en la nube (Groq) es inmediata, la vectorización local (embeddings) constituye un cuello de botella en CPUs estándar (latencia de ~40s). Esto permite concluir que, para un

despliegue productivo, es imperativo el uso de aceleración por GPU, validando la necesidad de arquitecturas híbridas o totalmente en la nube.

11.3. Impacto y proyección

Este proyecto demuestra que la tecnología actual permite "humanizar" el acceso a las bases de datos. Ya no es necesario que el usuario aprenda a buscar; es el sistema el que ha aprendido a entender.

En definitiva, la solución desarrollada no es simplemente un buscador de series, sino una prueba de concepto de cómo los **Asistentes Cognitivos** pueden actuar como intermediarios eficaces entre grandes volúmenes de datos y las necesidades humanas, ofreciendo una experiencia de usuario personalizada, fluida y, sobre todo, útil.

Bibliografía

[Documentación oficial de Uvicorn](#)

[Documentación oficial de FastAPI](#)

[Documentación oficial de LlamaIndex](#)

[Documentación oficial de Pandas](#)

[Documentación oficial de Grok](#)

[Documentación oficial de Ollama](#)

[Documentación oficial de Python](#)

[Documentación oficial de Neo4j](#)

[Google Cloud - Documentación sobre RAG](#)

[LlamaIndex - Documentación sobre RAG](#)

[IBM - Documentación sobre RAG](#)

[IBM - Patrones de diseño RAG](#)

[Microsoft - Patrones de diseño RAG](#)

[Youtube - NODES 2023 - Using LLMs to Convert Unstructured Data to Knowledge Graphs](#)

[Youtube - LlamaIndex Webinar: Advanced RAG with Knowledge Graphs \(with Tomaz from Neo4j\)](#)

[Youtube - Local GraphRAG with LLaMa 3.1 - LangChain, Ollama & Neo4j](#)

[Youtube - Curso de Ollama: TODO lo que Necesitas Saber para una IA PRIVADA y segura](#)

[Youtube - Python Advanced AI Agent Tutorial - LlamaIndex, Ollama and Multi-LLM!](#)

[Youtube - ¿Qué es RAG? - Clase con código y ejemplo](#)

[Youtube - Neo4j \(Graph Database\) Crash Course](#)

[Kaggle - Dataset de entrenamiento](#)